

What are Angular Forms?

Forms in Angular are used to **capture user input** (like text, email, password, checkbox, etc.), **validate it**, and then **use it in your app** (send to an API, store, etc.).

Angular provides **two approaches** to build forms:

1. **Template-driven forms** – simple, suitable for small forms, use Angular directives in the HTML.
2. **Reactive forms (Model-driven forms)** – powerful, suitable for complex and dynamic forms, use explicit FormControl and FormGroup in TypeScript.

◆ 1. Template-driven Forms

- Built mainly in the HTML template.
- Uses Angular directives like ngModel, ngForm.
- Easier but less scalable.

👉 Example:

```
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
```

```
<label>Name:</label>
```

```
<input type="text" name="name" ngModel required>
```

```
<label>Email:</label>
```

```
<input type="email" name="email" ngModel required email>
```

```
<button type="submit" [disabled]="userForm.invalid">Submit</button>
```

```
</form>
```

👉 Component:

```
onSubmit(form: any) {
```

```
  console.log(form.value); // { name: "John", email: "john@example.com" }
```

```
}
```

✓ **Pros:** Simple, less code.

✗ **Cons:** Harder to unit test, less control for dynamic forms.

◆ 2. Reactive Forms

- Built mainly in **TypeScript**.
- Uses FormControl, FormGroup, and FormBuilder.
- More structured and testable.

👉 Example:

```
import { Component } from '@angular/core';  
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
@Component({  
  selector: 'app-user-form',  
  templateUrl: './user-form.component.html'  
})  
export class UserFormComponent {  
  userForm = new FormGroup({  
    name: new FormControl('', Validators.required),  
    email: new FormControl('', [Validators.required, Validators.email])  
  });  
  
  onSubmit() {  
    console.log(this.userForm.value);  
  }  
}
```

👉 Template:

```
<form [formGroup]="userForm" (ngSubmit)="onSubmit()">
  <label>Name:</label>
  <input type="text" formControlName="name">

  <label>Email:</label>
  <input type="email" formControlName="email">

  <button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>
```

✓ **Pros:** Scalable, reusable, easier to validate & test.

✗ **Cons:** More boilerplate than template-driven.

◆ 3. Validation in Angular Forms

Both approaches support validation:

- **Built-in validators:** required, minLength, maxLength, email, etc.
- **Custom validators:** You can create your own logic.
- **Async validators:** e.g., checking if a username already exists from a server.

👉 Example (Reactive):

```
email: new FormControl("", [
  Validators.required,
  Validators.email
])
```

👉 Example (Template-driven):

```
<input type="email" name="email" ngModel required email>
```

◆ 4. Key Directives & Classes

- **Template-driven:**
 - `ngForm` → tracks form state
 - `ngModel` → two-way data binding
 - `required`, `minlength`, `email` → validators
- **Reactive:**
 - `FormGroup` → group of controls
 - `FormControl` → individual input
 - `FormArray` → array of controls (dynamic forms)
 - `FormBuilder` → helper service to create forms faster

◆ When to Use What?

- **Template-driven** → Small, simple forms (like login, contact form).
- **Reactive** → Complex, dynamic, large forms (like multi-step forms, e-commerce checkout).

✅ In short:

- Angular forms help manage **input, validation, and state**.
- You can choose between **template-driven (simple)** and **reactive (structured & powerful)**.

• Side-by-side Summary

Feature	Template-driven	Reactive
Setup	Mostly HTML	Mostly TypeScript
Control	Implicit (<code>ngModel</code>)	Explicit (<code>FormControl</code>)
Validation	Directives (<code>required</code> , <code>email</code>)	Validators (<code>Validators.required</code>)
Best For	Small/simple forms	Complex/dynamic forms